DATA 612 · Statistical Programming in R · Week 1

1.1 Getting Started with R and RStudio

Install R and RStudio, run a script, and keep files where R can find them.

1. Install R, then RStudio

R is the language. RStudio (Posit) is an editor that makes R easier to use.

Install them in this order:

1. R for Windows, macOS, or Linux.
2. RStudio Desktop.

RStudio will not work properly if R is not installed first.

We will work in RStudio, not in the older R application that installs with R.

R was designed for statistics. It is free, it is built around packages, and it is a good language for analysis you can rerun. Python is also widely used in data science. This course uses R.

---

2. The RStudio Layout

When you open RStudio, you will typically see several panes:

- the **console**, where R evaluates code immediately
- a **source** editor, where you write scripts and documents you want to keep
- an **environment / history** pane, which lists objects in the current session
- a **files / plots / packages / help** pane

The source pane stays closed until you open or create a file.

A few habits that save time:

- The **Files** tab is a limited file manager. You can open, copy, and delete files, and you can set the console working directory from **More**.
- The **Environment** tab shows objects you have created in this session.
- The **Help** tab shows documentation. You can also type `help(mean)` or `?mean` in the console.

---

3. Configure RStudio Once

Open **Tools** → **Global Options…** → **General**.

Under **Workspace**:

- uncheck **Restore .RData into workspace at startup**
- set **Save workspace to .RData on exit** to **Never**

If RStudio restores yesterday's objects, you can accidentally use results you no longer have code for. That looks convenient and is bad for reproducibility.

Other useful options:

- **Code** → **Editing**: turn on soft wrapping
- **Appearance**: choose a font size you can read

---

4. The Console

The console is the pane with the `>` prompt.

Type:

```
1 + 1
```

and press Enter.

R prints:

```
# [1] 2
```

The `[1]` is not part of the answer. It is the index of the first element R is showing you.

The up and down arrow keys cycle through recent console commands. That is useful when you mistype a line.

R is a calculator:

```
3 * 7
9 / 3
(3 + 5) * 6
3 ^ 2
```

`%%` is remainder after integer division:

```
1 %% 2
# 1

4 %% 2
# 0
```

The console is useful for short experiments. It is a poor place to keep your work. When you close RStudio, console history is not a script.

If a command is incomplete, R shows a `+` prompt and waits. Press **Esc** to get back to `>`.

---

5. Scripts

A **script** is a text file of R code, usually with a `.R` extension.

Create one with **File** → **New File** → **R Script**.

Type this into the script:

```
x <- 10
x * 2
```

Place the cursor on a line and run it with **Ctrl+Enter** (Windows/Linux) or **Cmd+Enter** (macOS). You can also click **Run**.

The code is sent to the console, and the console shows the result.

A useful habit is:

> Type in a script. Run from the script. Treat the console as a scratch pad.

Save the script. You can reopen it in a later session.

Lab 1 is a script. R Markdown and Quarto are for later assignments; see note **1.4**.

---

6. Comments

R ignores anything after `#` on a line:

```
# This line is only for humans.

x <- 10   # store 10 in x
```

Comments are how you explain *why* you wrote something, not how you hide code you might need later.

---

7. Assignment

We store a value with `<-`:

```
greeting <- "Hello"
greeting
```

In this course:

- `<-` stores a value in an object
- `=` names an argument inside a function call, for example `mean(x, na.rm = TRUE)`

R is **case sensitive**. `x` and `X` are different objects. `TRUE` is not `true`. `sum()` is not `Sum()`.

---

8. Getting Help

Almost every function has a help page:

```
help(mean)
?mean
help("%%")
```

`??topic` searches help pages. Adding `"in R"` to a web search is often faster than memorizing function names.

---

9. Organize Your Files

Do not keep every course file on the Desktop with names like `file 1`. You will not be able to tell R where the data is, and other people will not be able to rerun your work.

Create one folder for the course, for example `data_612`, and keep lecture work and assignments in separate subfolders. Use **snake_case** names. Avoid spaces and special characters.

```
data_612/
  students.csv
  lab01.R
  data/
    students.csv
    extra/
  R/
  output/
  assignments/
    lab_01/
```

Inside a project, a common pattern is:

- `R/` for `.R` scripts
- `data/` for files you import
- `output/` for files you write

You do not need every folder every week. You do need a place that is not a pile.

Back up that folder to Google Drive or another cloud service. A crashed laptop is a common way to lose a course.

---

10. Where Are Your Files?

R does not search your whole computer for a file. It looks in one folder, called the **working directory**, unless you tell it a path.

```
getwd()
```

That path is "where R is standing." In RStudio you can also read it just under the Console tab.

In RStudio you can change it with **Session** → **Set Working Directory**, or in the **Files** pane: go to the folder, then **More** → **Set As Working Directory**.

A better habit for the course is **File** → **New Project**. An RStudio project pins the working directory to that folder. When the project is open, `getwd()` is the project folder. Do not nest one project inside another.

Absolute paths versus relative paths

A **path** is the address of a file.

An **absolute path** starts from the top of the drive or from your user folder:

```
"C:/Users/you/Documents/data_612/students.csv"
"/Users/you/Documents/data_612/students.csv"
```

It works only on *your* computer. If your username is in the path, nobody else can run the code — including whoever grades it. Do not put absolute paths in work you submit.

A **relative path** starts from the working directory. Think of it as directions from the house you are already in, not from the city.

Use `/` in paths in R, even on Windows. Do not use `\`.

How to read a relative path

Three pieces do all the work:

Piece	Meaning
`file.csv` or `./file.csv`	the file is in the working directory
`folder/file.csv`	go **down** into `folder`, then take the file
`../file.csv`	go **up** one folder, then take the file

`.` means "this folder." `..` means "the parent folder." You can stack them: `../../` goes up twice. You can go up and then down: `../other_folder/file.csv`.

The RStudio **Files** pane shows `..` at the top of a folder. That is the same `..` you write in a path.

A concrete folder tree

Suppose this is an RStudio project, and `getwd()` is `data_612`:

```
data_612/                ← working directory
  students.csv
  lab01.R
  data/
    students.csv
    extra/
      midterm.csv
  R/
    import_students.R
  output/
  assignments/
    lab_01/
      lab01.R
```

From this working directory, these paths all make sense:

```
"students.csv"                      # the copy in data_612/
"./students.csv"                    # the same file
"lab01.R"
"data/students.csv"                 # one folder down
"data/extra/midterm.csv"            # two folders down
"R/import_students.R"
"assignments/lab_01/lab01.R"
```

So:

```
read.csv("students.csv")
read.csv("data/students.csv")
read.csv("data/extra/midterm.csv")
```

read three different files. The first is next to the project. The second is in `data/`. The third is in `data/extra/`.

To write a file into `output/`:

```
write.csv(students, "output/grades.csv")
```

Same tree, different working directory

Relative paths change if you move where R is standing.

If you set the working directory to `data_612/data`, then `getwd()` ends in `.../data_612/data`, and the same files look like this:

```
"students.csv"                      # now this is data/students.csv
"extra/midterm.csv"
"../students.csv"                   # up to data_612/, then that copy
"../lab01.R"
"../R/import_students.R"
"../assignments/lab_01/lab01.R"
"../output/grades.csv"
```

If you set the working directory to `data_612/assignments/lab_01`:

```
"lab01.R"                           # the copy in lab_01/
"../.."                             # that path is the project folder itself
"../../students.csv"                # up to assignments/, up to data_612/
"../../data/students.csv"
"../../data/extra/midterm.csv"
"../../R/import_students.R"
```

`../` once leaves `lab_01` and lands in `assignments`. `../../` leaves `assignments` and lands in `data_612`.

Check that R sees the file

Before you import, ask R what it can see:

```
getwd()
list.files()
list.files("data")
file.exists("students.csv")
file.exists("data/students.csv")
```

`list.files()` lists the working directory. `file.exists()` is `TRUE` only if that relative path is right from *here*.

If `read.csv("students.csv")` fails, the usual reasons are:

- the file is not in the working directory (it is in `data/`, or still in Downloads)
- the name is slightly wrong (`Students.csv` is not `students.csv` on some systems)
- you used an absolute path that does not exist on this computer
- you used `\` instead of `/`

Move the file, or fix the relative path. Do not paste a `C:/Users/...` path into the script.

Note **1.2** shows how to import `students.csv`. Lab 1 asks you to import it. Put that file in the working directory, or use a relative path such as `"data/students.csv"` if you keep data in a subfolder.

---

11. Errors Are Information

If R cannot run a line, it prints an error. Read it from the top.

Common first-week mistakes include:

- a missing quotation mark
- a missing parenthesis
- running `install.packages(ggplot2)` without quotes
- calling a function from a package you have not loaded
- looking for a file that is not in the working directory

You do not need to memorize every error. You do need to read it.

---

12. What to Do Next

Once R and RStudio are installed and you can run a line from a script, continue with:

- **1.2** R Packages and the Tidyverse, including how to import a CSV
- **1.3** Introduction to R Concepts
- **1.4** R Markdown and Quarto
- **Lab 1**, which asks you to predict what R will do *before* you run the code. Work in a `.R` script.